Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Puebla

TE3001B. Fundamentación de robótica (Gpo 101)

Reto semanal 1. Manchester Robotics

Alumnos:

- Bruno Alejandro Domínguez Maldonado | A01738844
- Iván Ariel Rebollar León | A01737891
- Omar Kuri Vázquez | A01738129
- Erik García Cruz | A01732440

13 de abril de 2026

Resumen

Este reporte documenta el desarrollo de un sistema de navegación autónoma para el robot diferencial Puzzlebot, operando en un ecosistema de ROS 2. La investigación comenzó con el despliegue de una infraestructura inalámbrica mediante el protocolo SSH y la implementación del puente micro-ROS para la comunicación serial con el microcontrolador. Un componente central de este trabajo fue la caracterización de los actuadores, identificando la región lineal, la zona muerta y los límites de saturación para garantizar la integridad del hardware. Se diseñó una arquitectura de software compuesta por dos nodos: un Generador de Trayectorias que procesa coordenadas cartesianas y un Controlador de Lazo Abierto regido por una Máquina de Estados Finitos (FSM). La precisión del sistema fue optimizada mediante la calibración de constantes de ganancia (Kp) para compensar la fricción y la inercia mecánica obtenidas mediante odometria. El éxito del sistema se validó mediante la ejecución de una trayectoria de cuatro puntos, analizando los registros de tiempo y velocidad en un entorno de autonomía estricta (Open Robotics, 2024; Siegwart et al., 2011).

Objetivos

Objetivo General

- Desarrollar un ecosistema de software modular en ROS 2 que permita al robot Puzzlebot realizar trayectorias autónomas precisas en lazo abierto, optimizando la respuesta mecánica mediante la caracterización de sus componentes electrónicos y la calibración de software.

Objetivos Específicos

- Configurar la Infraestructura de Red: Establecer una comunicación remota vía SSH entre la estación de control y la NVIDIA Jetson Nano, garantizando una latencia mínima en la publicación de tópicos.
- Caracterizar el Sistema de Actuación: Identificar experimentalmente la señal mínima de activación (dead zone) y el punto de saturación de los motores para definir un rango operativo seguro.

- Implementar Mensajería Personalizada: Crear e integrar la interfaz CustomPose para transportar datos de posición, velocidad y tiempo entre nodos de planificación y ejecución.
- Desarrollar Lógica de Control de Estados: Programar una FSM que gestione las transiciones entre reposo, giro y avance, utilizando temporizadores de alta resolución.
- Calibrar Parámetros Cinemáticos: Ajustar las ganancias de corrección lineal y angular (Kp_lineal, Kp_giro) para minimizar el error acumulado por factores físicos no modelados.

Introducción

La robótica móvil diferencial basa su movimiento en la manipulación independiente de la velocidad de dos ruedas situadas en un eje común. En el caso del Puzzlebot, la inteligencia reside en una NVIDIA Jetson Nano, que actúa como el nodo maestro en ROS 2, mientras que la ejecución de potencia se delega a una placa Hackerboard (Quigley et al., 2015).

Mensajes Pose, Twist y CustomPose

En el estándar de ROS 2, la comunicación se realiza mediante interfaces. Los mensajes geometry_msgs/Twist definen la velocidad cinemática a través de dos vectores:

Lineal (v): Velocidad en el eje x (m/s).

Angular (ω): Velocidad de rotación sobre el eje z (rad/s).

Sin embargo, para una navegación autónoma por puntos, el robot necesita conocer la "meta" completa. Mientras que geometry_msgs/Pose ofrece posición y orientación, el proyecto exigió la creación de CustomPose, que añade los campos target_velocity y target_time. Esta estructura permite que el nodo de control no solo sepa a dónde ir, sino a qué ritmo debe hacerlo para mantener la estabilidad del chasis (Lalancette et al., 2024).

Fundamentos del Control en Lazo Abierto

Un sistema de control en lazo abierto (open-loop) opera sin utilizar información de retroalimentación de la salida para modificar la entrada de control. El controlador asume que, dada una instrucción precisa, la planta (el robot) ejecutará el movimiento de manera perfecta. Sin embargo, en la robótica móvil física, esto es altamente susceptible a errores debido a

perturbaciones externas, no lineales y ruido en los actuadores. La justificación de implementar un esquema en lazo abierto en esta fase radica en la necesidad de comprender profundamente la dinámica del sistema antes de introducir complejos filtros o controladores PID.

Cinemática Directa: Cálculo de Distancia y Ángulo

El cálculo del desplazamiento se rige por integrales simples del movimiento.

Para la traslación lineal en un intervalo de tiempo t:

$$d = \int_0^t v(\tau)d\tau \approx v \cdot t$$

Para la rotación sobre su propio centro instantáneo de rotación:

$$\theta = \int_0^t \omega(\tau)d\tau \approx \omega \cdot t$$

Estas ecuaciones permiten despejar el tiempo necesario si el usuario provee una velocidad, o calcular la velocidad requerida si el usuario impone una restricción temporal estricta.

Caracterización de los Actuadores: Zona Muerta, Linealidad y Saturación

Los motores DC no responden de manera ideal a las señales de voltaje debido a parámetros provenientes de la propia constitución de los mismos, parámetros tales como la fricción de los engranajes de los motores . Su comportamiento se divide en tres zonas críticas:

Dead Zone (Zona Muerta): Es el rango de señales de entrada (usualmente de 0 a un umbral PWMmin) donde el par electromagnético generado por las bobinas no es suficiente para vencer la fricción estática de la caja reductora, los cojinetes y el peso del propio robot contra el suelo. En esta zona, la entrada es mayor a cero, pero la velocidad de salida es nula.

Zona Lineal: Una vez superada la dead zone, el motor entra en su región de funcionamiento nominal. Aquí, existe una relación aproximadamente proporcional y lineal entre el incremento de la señal de control y el incremento de la velocidad angular del eje del motor.

Saturación: Ocurre cuando el motor ha alcanzado su límite físico y eléctrico. Aunque el controlador intente enviar una señal mayor (acercándose al 100% del PWM o voltaje máximo de la batería), la velocidad angular no aumenta debido a las limitaciones constructivas del estator y las fuerzas contraelectromotriz.

Solución del Problema

Metodología

La implementación se desglosó en cuatro fases críticas, diseñadas para para transformar la plataforma en un sistema de navegación autónoma funcional

Fase 1: Infraestructura de Comunicación y Túnel de Datos

Se configuró la NVIDIA Jetson Nano como un nodo maestro bajo una arquitectura de red inalámbrica Hotspot.

Protocolo SSH: Se estableció una sesión remota segura mediante ssh puzzlebot@10.42.0.2. Para garantizar que los procesos no se interrumpieran ante una eventual pérdida de señal Wi-Fi, se utilizaron multiplexores de terminal, asegurando la persistencia de la ejecución de ROS 2.

Capa micro-ROS: Se desplegó el micro_ros_agent utilizando una conexión serial de alta velocidad (115200 baudios) hacia el microcontrolador. La robustez en esta etapa se validó mediante el monitoreo de la frecuencia de publicación de los tópicos /VelEncL y /VelEncR. Se determinó que cualquier caída en la frecuencia por debajo de los 20Hz introducía errores inaceptables en la estimación de tiempo para el lazo abierto (Open Robotics, 2024).

Fase 2: Caracterización Electromecánica y Análisis de la Región Lineal

Identificación de la Zona Muerta (Dead Zone): Mediante un Teleop de prueba, se enviaron comandos incrementales de velocidad en pasos de 0.005 m/s. Se observó que, debido a la fricción estática y el peso del robot, el desplazamiento real comenzaba solo a partir de los 0.05 m/s. Este valor se estableció como el límite inferior de seguridad en el código. Estando la zona muerta entre 0 m/s y 0.05 m/s.

Análisis de Saturación y Anti-Brownout: Se identificó que por encima de los 0.45 m/s, el consumo de corriente de los motores provocaba caídas de tensión (brownouts) que afectaban la estabilidad de la Jetson Nano. Por tanto, se programó un filtro de saturación estricto en el nodo path_generator que limita cualquier comando a un Vmax=0.5 m/s y Wmax=0.5 rad/s (Siegwart et al., 2011).

Fase 3: Algoritmia

Trayectoria Cuadrada

Para validar la respuesta dinámica del Puzzlebot y la precisión del control en lazo abierto, se implementó un nodo específico orientado a completar un cuadrado de 2 X 2 metros. A diferencia del generador de trayectorias general, este nodo utiliza una Máquina de Estados Finitos (FSM) simplificada para alternar entre desplazamientos lineales y giros puntuales.

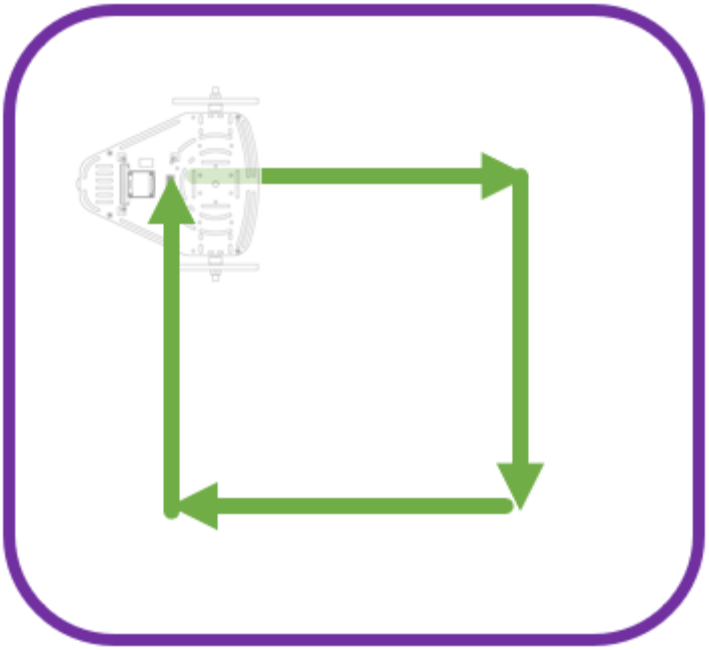

Figura 1: trayectoria cuadrada de 2 X 2 metros que debía seguir el puzzlebot en la primera parte del reto

Se definió la robustez como la capacidad del sistema para minimizar el error acumulado derivado de la fricción superficial y la inercia del robot. Para lograrlo, el controlador implementa:

- Auto-tuning de tiempos: El usuario define la velocidad lineal (v) y angular (w), y el nodo calcula automáticamente los tiempos de ejecución basados en la distancia objetivo (d = 2m) y el ángulo ($\theta = \pi/2$).
- Compensación por Ganancias (K_p): Se introdujeron constantes de calibración para ajustar el comportamiento real frente al teórico:
    - Kp_lineal = 0.93$: Corrige el sobrepaso por inercia en tramos rectos.

- Kp_giro = 0.9375$: Ajusta la fricción de las llantas durante la rotación para asegurar giros de 90° exactos.

Para la arquitectura de la máquina de estados (FSM) dentro del control se rige por tres estados principales:

- Estado 0 (Recto): Pública linear.x = 0.2 m/s hasta que t >= (d/v) * Kp_lineal
- Estado 1 (Giro): Pública angular.z = 0.5 rad/s hasta que t >= (Ɵ/w) * Kp_giro
- Estado 2 (Parada): Envía comandos de velocidad cero y finaliza el nodo tras completar 4 iteraciones.

Con esta lógica se completa el cuadrado requerido. A continuación la máquina de estados del código:

```
if self.state == 0: # AVANZAR

    cmd.linear.x = self.linear_speed

    if elapsed_time >= self.forward_time:

        self.state = 1

        self.state_start_time = now

elif self.state == 1: # GIRAR

    cmd.angular.z = self.angular_speed

    if elapsed_time >= self.rotate_time:

        self.side_count += 1

        self.state = 0 if self.side_count < 4 else 2

        self.state_start_time = now
```

De esta forma el código quedó estructurado con un solo nodo open_loop_ctrl_2.p con el que

se emplea el controlador de lazo abierto y la máquina de estados simplificada necesaria para el cuadro de 2 X 2 metros.

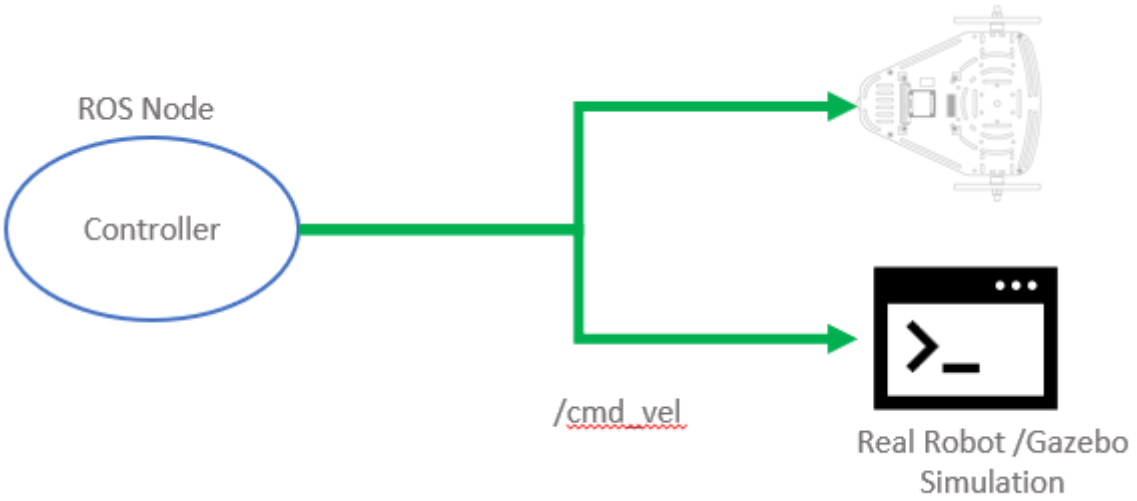

Figura 2: diagrama del nodo que controla al puzzlebot para completar el cuadrado.

Generador de Trayectorias (path_generator.py)

El diseño del planificador se centró en la precisión geométrica y la optimización del movimiento utilizando exclusivamente NumPy para operaciones vectoriales.

Para la transformación de coordenadas en cada punto Pk(x,y), se calculó el vector de desplazamiento d. El ángulo requerido se obtuvo mediante la función atan2, la cual es intrínsecamente más robusta que atan al manejar correctamente los cuatro cuadrantes del plano cartesiano.

En este tenor, para la lógica de "Wrap to Pi" para evitar giros ineficientes (por ejemplo, girar 270° a la derecha en lugar de 90° a la izquierda), se implementó una normalización angular

```
result = np.fmod((theta + np.pi), (2 * np.pi))

if result < 0: result += 2 * np.pi

return result - np.pi
```

Sincronización Dinámica de Nodos: El nodo calcula el tiempo total estimado (Ttotal=Tgiro+Tavance+1.0s). Este valor se inyecta en el periodo del Timer de ROS 2,

permitiendo que el sistema espere a que el robot complete físicamente la maniobra antes de publicar la siguiente meta en el tópico /pose.

Fase 4: Controlador Maestro y Máquina de Estados (open_loop_ctrl_2.py)

El controlador se diseñó como una Máquina de Estados Finitos (FSM) síncrona, eliminando cualquier ambigüedad en la ejecución.

Estado 0 (Giro): El robot gira sobre su centro instantáneo de rotación. Se observó experimentalmente un fenómeno de sobregiro debido a la inercia del chasis. Para mitigar esto, se calibró la constante self.kp_giro = 0.84. Esta ganancia ajusta el tiempo de rotación para compensar el tiempo que tarda el motor en detenerse totalmente tras enviar el comando cero.

Estado 1 (Avance): El robot se desplaza en línea recta. Se implementó la ganancia self.kp_lineal = 0.88. Este ajuste de robustez asegura que el robot recorra la distancia exacta, compensando el deslizamiento cinético que ocurre en superficies lisas.

Estado 2 (Stop): Un estado de transición de 0.1s que limpia los buffers de velocidad, asegurando que el robot esté en reposo absoluto antes de iniciar el siguiente giro, lo que previene la acumulación de errores por fuerzas centrífugas (Quigley et al., 2015).

Estado 3 (Idle): Suscripción pasiva al mensaje CustomPose. Al recibir datos, extrae la velocidad y el tiempo calculados, aplicando inmediatamente los factores de corrección.

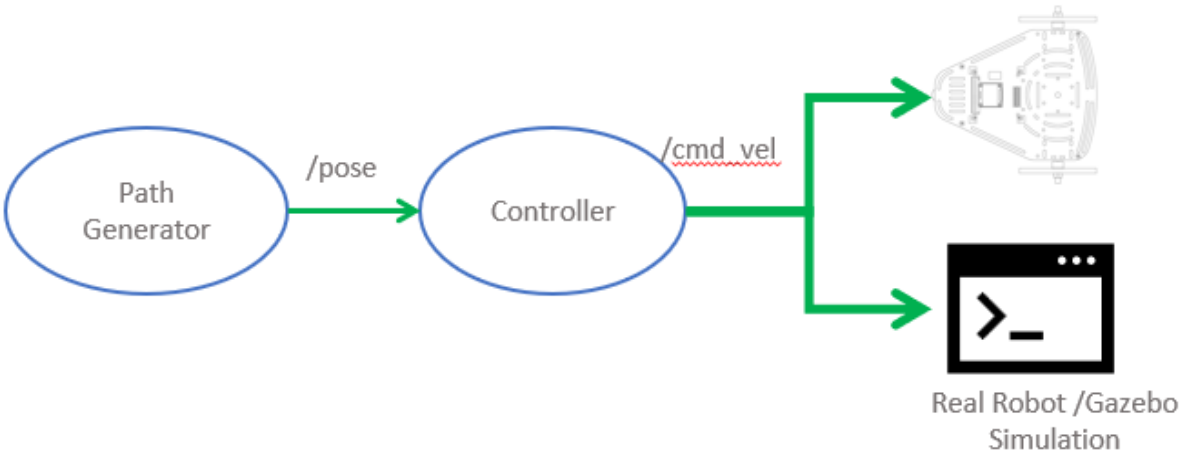

Figura 3: diagrama de la forma de comunicación entre el nodo generador de señal y el nodo de control mediante el topico /pose. Debido a que no se nos solicitó, no empleamos simulación en Gazebo.

Análisis del código:

El desarrollo del sistema se realizó mediante una arquitectura modular basada en dos nodos principales en ROS 2, con el objetivo de separar la planificación de trayectorias de la ejecución física del movimiento.

Nodo path_generator.py

Este nodo se encarga de la planeación geométrica de la trayectoria. Su función principal consiste en recibir las coordenadas objetivo (x,y), calcular la distancia al siguiente punto y determinar el ángulo necesario para orientar el robot correctamente.

Para ello se emplearon las siguientes funciones principales:

- atan2(dy, dx) para calcular el ángulo objetivo
- wrap_to_Pi() para normalizar el ángulo del intervalo [−π,π]
- enviar_siguiente_punto() para publicar secuencialmente cada meta

La distancia se calcula mediante:

$$d = \sqrt{(x_f - x_i)^2 + (y_f - y_i)^2}$$

Mientras que la corrección angular se realiza mediante la función:

*result = np.fmod((theta + np.pi), (2 * np.pi))*

if result < 0:

*result += 2 * np.pi*

return result - np.pi

Esta lógica permite evitar giros innecesarios y optimizar el recorrido.

Una vez calculados los parámetros, el nodo empaqueta la información dentro del mensaje *CustomPose*, incluyendo:

- posición objetivo
- ángulo requerido
- velocidad objetivo

- tiempo estimado

Posteriormente, el mensaje se publica en el tópico */pose.*

Nodo open_loop_ctrl_2.py

Este nodo corresponde al controlador de movimiento del robot. Su funcionamiento está basado en una Máquina de Estados Finitos (FSM) que organiza el movimiento en diferentes etapas.

Los estados implementados fueron:

- Estado 3: espera de nueva meta
- Estado 0: giro
- Estado 1: avance lineal
- Estado 2: parada

Al recibir el mensaje *CustomPose*, el controlador extrae la información necesaria y comienza la ejecución de la trayectoria.

Primero realiza el giro utilizando una velocidad angular constante, y posteriormente avanza en línea recta con la velocidad indicada.

Para mejorar la precisión experimental, se implementaron constantes de calibración:

$$K_{p,giro} = 0.84$$

$$K_{p,lineal} = 0.88$$

las cuales compensan errores derivados de la inercia, la fricción y el deslizamiento del chasis.

Finalmente, el nodo publica los comandos de velocidad en el tópico /cmd_vel, que son interpretados por el robot para ejecutar el movimiento físico.

Resultados

Primera parte: recorrido con el proyecto de un solo nodo

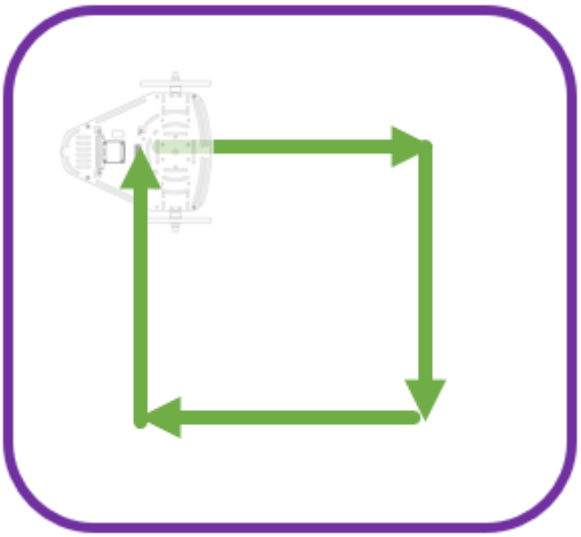

Figura 4. Primer recorrido realizado por el Puzzlebot.

Video del recorrido (primer código):
https://drive.google.com/file/d/1wBLO6aIvQrxMIOqXf95TJFQZTm4IYRGV/view?usp=sharing

En el video se aprecia el cumplimiento del recorrido autónomo del Puzzlebot mediante una trayectoria cuadrangular. Se observa que el robot ejecuta giros precisos de 90° y desplazamientos lineales constantes, validando la correcta calibración de las K_p y la gestión de la máquina de estados.

En esta prueba se utilizó código con el nodo único open_loop_ctrl.py, cuya lógica está basada en una máquina de estados de tres etapas: avance, giro y parada. Para el desplazamiento se utilizó una velocidad lineal constante de 0.2 m/s, mientras que para el giro se empleó una velocidad angular de 0.5 rad/s.

El tiempo de avance fue calculado a partir de la distancia objetivo mediante la relación:

$$t = \frac{d}{v} \cdot K_{p,lineal}$$

empleando:

$$K_{p,lineal} = 0.93$$

Por su parte, el tiempo de giro se calculó con:

$$t = \frac{\theta}{\omega} \cdot K_{p,giro}$$

donde

$$K_{p,giro} = 0.9375$$

A pesar de operar en lazo abierto, el sistema muestra una alta repetibilidad y un error de cierre mínimo, lo que confirma que la caracterización de la zona muerta y los límites de saturación permiten compensar adecuadamente los efectos físicos de fricción e inercia del chasis.

Segunda parte: (proyecto con dos nodos)

Primer recorrido

Video del recorrido (segundo código):
https://drive.google.com/file/d/17FgdNO_hac6dWbPdL5RI7CX7WvfGM6kh/view?usp=sharing

Así como en el video previo, en este se puede apreciar al Puzzlebot realizando un recorrido con forma de cuadrilátero, más cercano a un rectángulo que a un cuadrado.

En esta prueba se utilizó la arquitectura basada en dos nodos: path_generator.py y open_loop_ctrl_2.py.

El nodo generador calcula:

- distancia al siguiente punto
- ángulo requerido
- velocidad objetivo
- tiempo estimado

mientras que el nodo controlador ejecuta la trayectoria mediante una FSM de cuatro estados:

- espera

- giro
- avance
- parada

Las ganancias utilizadas fueron:

$$K_{p,lineal} = 0.88$$

$$K_{p,giro} = 0.84$$

lo que permitió mejorar la precisión del giro y reducir el error acumulado por deslizamiento.

Video de la pantalla junto con los inputs:
https://drive.google.com/file/d/1LN6xbpEoiJt2duyArbYKIiwja_OVv8tu/view?usp=sharing

En esta captura de video de la pantalla del ordenador se observan los comandos utilizados para ejecutar el código, así como los inputs requeridos por el nodo path_generator.py.

En este caso, se ingresaron:

- coordenadas cartesianas (x,y)
- velocidad objetivo
- puntos de trayectoria

A partir de estos valores, el sistema calcula automáticamente la distancia, el ángulo y el tiempo requerido para cada segmento.

Segundo recorrido

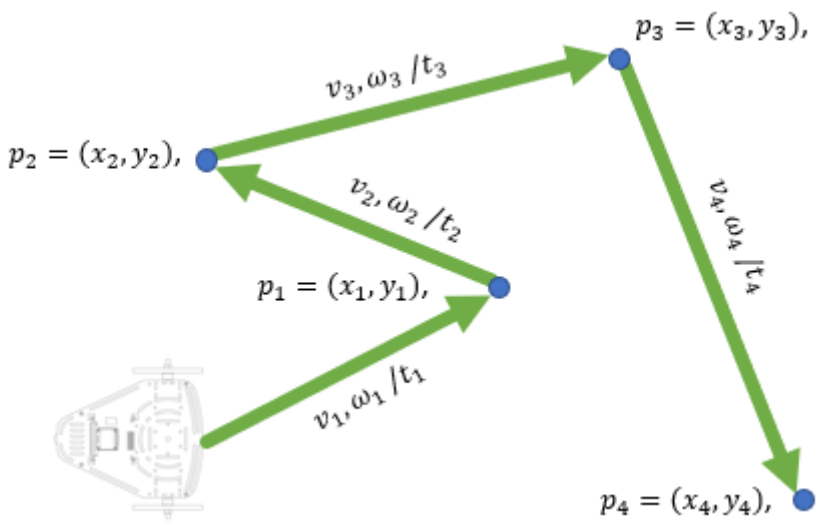

Figura 5. Segundo recorrido realizado por el Puzzlebot.

Video del recorrido utilizando velocidad:
https://drive.google.com/file/d/1rBv40ijm0n0xySVhAQ1TtnQ1SQuGCfsY/view?usp=sharing

En el video se observa al Puzzlebot ejecutando una trayectoria de navegación por puntos en zigzag, siguiendo la lógica de los vectores p_1 a p_4 mostrados en el diagrama.

Se aprecia cómo el robot realiza giros de orientación específicos antes de cada desplazamiento lineal para alcanzar las coordenadas cartesianas programadas.

En esta prueba, la variable principal utilizada fue la velocidad.

El usuario ingresó directamente la velocidad deseada para cada tramo, y el nodo path_generator.py calculó automáticamente el tiempo necesario utilizando:

$$t = \frac{d}{v}$$

Esto permitió que el robot ajustara dinámicamente la duración del movimiento en función de la distancia entre puntos.

La velocidad fue limitada por seguridad a:

$$v_{max} = 0.5\, m/s$$

mientras que la velocidad angular máxima se mantuvo en:

$$\omega_{max} = 0.5\, rad/s$$

Video de la pantalla al ejecutar el código junto con los inputs:
https://drive.google.com/file/d/1rvFJmcnfBT9ODCJeaYYnWTdZ1WEv43IU/view?usp=sharing

En este video, como en la captura anterior, se observan los comandos utilizados y los inputs ingresados para que el Puzzlebot recorra la trayectoria establecida.

En este caso, se ingresó la velocidad como parámetro principal, permitiendo al sistema calcular automáticamente el tiempo requerido para completar cada segmento.

Video del recorrido utilizando tiempo:
https://drive.google.com/file/d/11Gow7VP8_3KIL0GHUw776nqlrf_PfaAH/view?usp=sharing

En el video se observa la misma trayectoria que el Puzzlebot realizó en el video de demostración previo; no obstante, en esta ocasión se utilizó la variante en la que el usuario ingresa directamente el tiempo deseado.

La lógica implementada calcula la velocidad necesaria mediante:

$$v = \frac{d}{t}$$

lo que demuestra la flexibilidad del sistema para trabajar con dos tipos de consignas: velocidad o tiempo.

Video de la pantalla al ejecutar el código junto con los inputs:
https://drive.google.com/file/d/1yUfJEC3ePWUZwKEwS8zQZkOCid8N25DF/view?usp=sharing

En este video se observan nuevamente los comandos utilizados y los inputs ingresados para que el Puzzlebot recorra la trayectoria establecida.

A diferencia de la prueba anterior, en este caso se utilizó el tiempo como variable de entrada, obteniendo igualmente un recorrido satisfactorio y validando el correcto funcionamiento del generador de trayectorias.

Conclusiones

Tras el desarrollo de este reto, se validó que es posible lograr una navegación autónoma aceptable en el Puzzlebot utilizando un esquema de lazo abierto en ROS 2. La decisión de separar el sistema en dos nodos (planificador y controlador) fue fundamental, ya que permitió depurar la lógica de trayectorias sin interferir con la ejecución de potencia en los motores. Los objetivos se cumplieron en su mayoría, logrando que el robot completara recorridos en zigzag y cuadrados con una repetibilidad que superó las expectativas iniciales para un sistema sin retroalimentación.

La clave del éxito no estuvo solo en el código, sino en la caracterización experimental. Identificar la zona muerta y ajustar las constantes de ganancia (Kp_lineal de 0.88 y Kp_giro de 0.84) fue lo que realmente permitió compensar la inercia y la fricción del chasis. Sin estos ajustes manuales, el error acumulado por el deslizamiento de las ruedas habría hecho imposible que el robot regresara a su punto de origen.

No obstante, la experimentación también reveló limitaciones críticas. La más notable fue la inestabilidad del Hotspot de la NVIDIA Jetson Nano, que presentó micro-desconexiones de SSH durante las pruebas. Esto deja claro que, en robótica, la infraestructura de comunicación es tan vital como el algoritmo de control. Además, al no contar con encoders, el robot sigue siendo vulnerable a perturbaciones externas como irregularidades en el suelo.

Como aprendizaje final, queda claro que la navegación en lazo abierto es una excelente base para entender la dinámica del robot, pero para aplicaciones de mayor precisión es

indispensable evolucionar hacia un control en lazo cerrado. Implementar odometría y mejorar la robustez de la red inalámbrica serían los siguientes pasos naturales para transformar este prototipo en un sistema autónomo profesional.

Referencias

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825), 357-362. https://doi.org/10.1038/s41586-020-2649-2

Lalancette, C., Hendrix, A., & Jay, G. T. (2024). teleop_twist_keyboard. ROS 2 Documentation. Recuperado de https://docs.ros.org/en/ros2_packages/humble/api/teleop_twist_keyboard/index.html

micro-ROS. (s.f.). ROS 2 feature comparison. micro-ROS Documentation. Recuperado de https://micro.ros.org/docs/overview/ROS_2_feature_comparison/

Open Robotics. (2024). ROS 2 documentation: Humble Hawksbill. ROS 2 Project. Recuperado de https://docs.ros.org/en/humble/index.html

Quigley, M., Gerkey, B., & Smart, W. D. (2015). Programming Robots with ROS: a practical introduction to the Robot Operating System. O'Reilly Media, Inc.

Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). Introduction to autonomous mobile robots. MIT press.